



Distributed Data Storage: Replication & Fragmentation

Distributed databases store relations using two main approaches: **replication and fragmentation**.

Replication :

Several identical copies (replicas) of the relation are maintained.

Each replica is stored at a different site.

Fragmentation :

The relation is divided into smaller parts called fragments.

Each fragment is stored at a different site.

Fragmentation and Replication Combined :

The relation is partitioned into fragments.

Each fragment can have multiple replicas stored at different sites.

 by Nandana Narayan

Data Replication: Enhancing Availability

Availability

If one site fails, another site has the same data, so queries can still run.

Increased Parallelism

Multiple sites can handle read queries at the same time, improving speed.

Update Overhead

When data changes, all replicas must be updated to stay consistent.

Reduced Data Movement

More replicas mean a higher chance the data is already at the site where it's needed

Consistency Management

Extra effort is needed to keep all copies the same.

Example: In banking, account balance must match at all sites





Managing Replicas: Primary Copy Concept

Primary Copy

Choosing one replica as the primary copy simplifies update management and consistency enforcement.

Example: Banking System

Account data is associated with the site where the account was opened, serving as the primary copy for updates.

Data Fragmentation: Partitioning Relations

A relation r can be divided into smaller parts called fragments: $r_1, r_2, \dots, r_n$.

These fragments help in reconstructing the original relation r .

Horizontal Fragmentation

The relation is split by tuples, assigning each tuple to one or more fragments. Fragments can be reconstructed by union.

Vertical Fragmentation

The relation is split by attributes, each fragment includes a subset of attributes. Reconstruction uses natural join.

Horizontal Fragmentation

- **Horizontal fragmentation** partitions a relation into subsets of tuples based on predicates.
- For example, an account relation can be divided by branch, with each fragment containing accounts from a specific branch.

$$\begin{aligned} \text{account}_1 &= \sigma_{\text{branch_name} = \text{"Hillside"}}(\text{account}) \\ \text{account}_2 &= \sigma_{\text{branch_name} = \text{"Valleyview"}}(\text{account}) \end{aligned}$$

- The original relation can be reconstructed by **taking the union of all fragments**, ensuring no data loss.

$$r = r_1 \cup r_2 \cup \dots \cup r_n$$

Vertical Fragmentation (Decomposition)

- Vertical fragmentation splits a relation $r(R)$ by attributes, not rows.
- Attributes are grouped into subsets: $R_1, R_2, \dots, R_n$ such that: $R = R_1 \cup R_2 \cup \dots \cup R_n$
- Each fragment r_i of r is defined by $\pi_{R_i}(r)$
- The original relation is reconstructed by performing a **natural join** on the fragments, ensuring data integrity.

$$r = r_1 \bowtie r_2 \bowtie r_3 \bowtie \dots \bowtie r_n$$

Ensuring Reconstruction of Relations

To reconstruct the original relation from fragments, **primary-key attributes or superkeys must be included in each fragment**. This ensures that fragments can be joined correctly.

Adding a special attribute like a tuple-id can also help uniquely identify tuples across fragments, facilitating accurate reconstruction.

Transparency in Distributed Databases

Transparency means users don't need to know where or how data is stored and accessed.

Types of Transparency:

- **Fragmentation Transparency**

Users are unaware of how data is split into fragments.

- **Replication Transparency**

Users see only one version of data, even if it's replicated for performance or availability.

- **Location Transparency**

Users don't need to know the physical location of data; the system retrieves it automatically

Naming in Distributed Databases:

All data items (relations, fragments, replicas) must have unique names.

Problem:

Different sites might use the same name for different items.

Solution:

Use a central name server to manage names.

Allow aliases for easier user access, translated internally by the system.